

Projet Inception (42)

Plan d'action détaillé — sessions de 2 à 3 heures

Ce document sert de contexte à réutiliser dans le projet Claude : il reprend le plan global établi, le détail de la partie « Docker fondamentaux », ainsi qu'un rappel du sujet tel que partagé en conversation. À coller/joindre dans les instructions du projet pour que chaque nouvelle conversation (une par sous-sujet) parte du même contexte.

Rappel du sujet (extrait fourni en conversation)

Le sujet Inception de 42 demande de mettre en place une petite infrastructure composée de plusieurs services (NGINX, WordPress + php-fpm, MariaDB) sous des règles précises : le tout doit tourner dans une machine virtuelle, orchestré via Docker Compose, avec un service par container et une image par service portant le même nom que ce service.

Contraintes clés à respecter tout au long du projet :

- Un container = un service = un rôle (pas plusieurs process indépendants dans le même container).
- Pas de tag **latest** pour les images.
- Pas de **network: host**, pas de **--link**.
- Les containers doivent redémarrer automatiquement en cas de crash.
- PID 1 doit être un vrai process daemon (pas de bidouille type *tail -f* ou boucle infinie).
- Volumes nommés pointant vers **/home/login/data** sur l'hôte (persistance des données).
- NGINX comme unique point d'entrée, en TLS (TLSv1.2/1.3 uniquement), rien d'autre exposé en clair.
- Utilisation de secrets Docker (pas de mots de passe en dur dans les Dockerfiles/reno).

Note : ceci est un résumé reconstitué à partir de l'échange ; le texte intégral et officiel du sujet 42 (PDF fourni par l'école) reste la référence à consulter en cas de doute sur un détail précis.

Plan global du projet

Estimation totale : ~26-35h, soit 10-13 sessions de 2-3h.

Phase 1 — Comprendre Docker (fondamentaux)

~3 sessions (6-8h)

- Session 1 : concepts de base — image vs container, layers, Dockerfile (FROM, RUN, COPY, ENTRYPOINT/CMD), différence avec une VM.
- Session 2 : PID 1 et daemons — pourquoi pas de tail -f ni de boucle infinie en entrypoint, comprendre les process zombies.
- Session 3 : docker-compose, volumes, networks, secrets — syntaxe compose, named volumes vs bind mounts, réseaux bridge custom, secrets vs .env.

Phase 2 — Comprendre chaque service isolément

~2 sessions (4-6h)

- Session 4 : NGINX + TLS — config minimale, forcer TLSv1.2/1.3, certificats auto-signés (openssl).
- Session 5 : WordPress/php-fpm + MariaDB — communication fastcgi_pass, setup MariaDB en CLI, WP-CLI pour automatiser l'install.

Phase 3 — Architecture / conception

1 session (2-3h)

- Dessiner le schéma : nginx (443) → wordpress:9000 → mariadb:3306, réseau custom, volumes sur /home/login/data.
- Structure de dossiers : srcs/requirements//Dockerfile, conf/, tools/ ; contenu du .env ; secrets nécessaires.
- Écrire le Makefile (cibles build, up, down, clean, re).

Phase 4 — Implémentation, service par service

~4-5 sessions (10-12h)

- Session 6 : MariaDB seul — Dockerfile, script d'init SQL, tester en isolation (docker run).
- Session 7 : WordPress + php-fpm seul — Dockerfile, install via wp-cli scripté, socket TCP.
- Session 8 : NGINX seul — Dockerfile, conf TLS, connexion à WordPress.
- Session 9 : Assemblage docker-compose — réseau, volumes nommés, restart policy.
- Session 10 : secrets + .env + Makefile final — retirer tout mot de passe en dur, vérifier le .gitignore.

Phase 5 — Tests & debug

~2 sessions (4-6h)

- Config /etc/hosts ou DNS local pour login.42.fr, test bout-en-bout en HTTPS.
- Test de persistance des volumes (redémarrage container, données conservées).
- Test de restart automatique en cas de crash.
- Relecture contre la checklist du sujet (latest interdit, pas de network:host, pas de --link...).

Ordre délibéré pour l'implémentation : MariaDB → WordPress → NGINX, chaque service ajouté dépendant du précédent, pour valider une brique à la fois plutôt que de debugger trois containers en même temps.

Setup VM — Mac (UTM) puis postes de l'école

Le sujet exige que le projet entier tourne dans une VM. Plutôt que de coder en Docker natif sur macOS puis migrer, on travaille directement dans une VM dès le départ.

- Sur le Mac : installer **UTM** (gratuit, adapté Apple Silicon/Intel), image Debian ou Alpine (version stable), VM avec 2-4 Go de RAM.
- Dans la VM : installer Docker + Docker Compose, et Git.
- Édition du code : soit VS Code en Remote-SSH vers la VM, soit dossier partagé UTM + commandes docker lancées dans la VM.
- Un repo Git privé comme pont entre le Mac et les postes de l'école (commit/push depuis la VM Mac, pull côté école).
- À l'école : cloner le repo dans la VM déjà fournie ; tout se rebuild depuis les Dockerfiles, donc pas de dépendance à l'architecture locale (ARM vs x86).
- Point d'attention : ne jamais committer les secrets/.env — .gitignore dès le premier commit, transfert des secrets à part (clé USB ou questionnaire perso).

Détail — Phase 1 : Docker fondamentaux

Sous-sujet en cours de discussion dans une conversation dédiée du projet.

Session 1 — Concepts de base (2-3h)

- VM vs container : ce que chacun isole réellement, pourquoi un container est plus léger (partage du noyau de l'hôte au lieu d'embarquer un OS complet).
- Image vs container : l'image comme « recette figée », le container comme instance en cours d'exécution de cette recette.
- Le système de layers : construction couche par couche, intérêt pour le cache au build.
- Anatomie d'un Dockerfile : FROM, RUN, COPY, WORKDIR, EXPOSE, ENTRYPOINT/CMD (et la différence entre les deux derniers).
- Commandes de base : build, run, ps, images, exec, logs.

Session 2 — PID 1 et daemons (2-3h)

- Pourquoi le sujet interdit tail -f, les boucles infinies bricolées, bash en entrypt.
- Le rôle spécial de PID 1 sous Linux (gestion des signaux, reaping des process zombies).
- Lancer un vrai daemon en foreground : nginx -g "daemon off;", mysqld_safe, php-fpm -F.
- Ce qui se passe si PID 1 meurt (le container s'arrête, contrairement à une VM).

Session 3 — Docker Compose, volumes, networks, secrets (2-3h)

- Pourquoi Compose : orchestrer plusieurs containers au lieu de multiples docker run.
- Syntaxe docker-compose.yml : services, build context, depends_on.
- Volumes : named volumes vs bind mounts, pourquoi le sujet exige /home/login/data.
- Networks : réseau bridge custom vs network:host (interdit par le sujet).
- Secrets Docker vs simple .env — différence et pourquoi le sujet veut de vrais secrets.

Sous-sujets à traiter en conversations séparées

Chaque ligne correspond à une conversation dédiée dans ce projet Claude, pour garder chaque discussion focalisée.

#	Sous-sujet	Statut
1	Docker fondamentaux	En cours
2	NGINX + TLS	À venir
3	MariaDB	À venir
4	WordPress + php-fpm	À venir
5	Architecture globale & structure de dossiers	À venir
6	Assemblage docker-compose	À venir
7	Tests & debug final	À venir
8	Setup VM Mac → école	À venir